



INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA DO PIAUÍ
CAMPUS PICOS
TECNÓLOGO EM ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

LUIS HENRIQUE DE MOURA SANTOS

ESCALABILIDADE EM SISTEMAS WEB: UM ESTUDO ENTRE MONÓLITOS E
ARQUITETURAS DISTRIBUÍDAS

PICOS, PI
2026

LUIS HENRIQUE DE MOURA SANTOS

ESCALABILIDADE EM SISTEMAS WEB: UM ESTUDO ENTRE MONÓLITOS E
ARQUITETURAS DISTRIBUÍDAS

Projeto de pesquisa apresentado como exigência para aprovação na disciplina Elaboração de Projeto do Curso de Análise e Desenvolvimento de Sistemas do Instituto Federal de Educação, Ciência e Tecnologia do Piauí, Campus Picos.

Orientador: Prof. Esp. Jorge Luis Lima

LUIS HENRIQUE DE MOURA SANTOS

ESCALABILIDADE EM SISTEMAS WEB: UM ESTUDO ENTRE MONÓLITOS E
ARQUITETURAS DISTRIBUÍDAS

Projeto de pesquisa apresentado como exigência para aprovação na disciplina Elaboração de Projeto do Curso de Análise e Desenvolvimento de Sistemas do Instituto Federal de Educação, Ciência e Tecnologia do Piauí, Campus Picos.

Aprovada em ____/____/____.

BANCA EXAMINADORA:

Prof. Esp. Jorge Luis Lima(Orientador)
Instituto Federal do Piauí (IFPI)

Prof. Mestre Ciro Ferreira de Carvalho Junior
Instituto Federal do Piauí (IFPI)

Prof. Esp. Denis Costa Paiva
Instituto Federal do Piauí (IFPI)

PICOS, PI
2026

LISTA DE TABELAS

Tabela 1 – Cronograma de Atividades	18
---	----

LISTA DE ABREVIATURAS E SIGLAS

ABCOMM	Associação Brasileira de Comércio Eletrônico
ACID	Atomicity, Consistency, Isolation, Durability
API	Application Programming Interface
CPU	Central Processing Unit
DDR	Double Data Rate
GC	Garbage Collector
HTTP	Hypertext Transfer Protocol
I/O	Input/Output (Entrada e Saída)
JIT	Just-In-Time
JVM	Java Virtual Machine
LTS	Long Term Support
RAM	Random Access Memory
SaaS	Software as a Service
SSD	Solid State Drive
TCC	Trabalho de Conclusão de Curso
URL	Uniform Resource Locator

LISTA DE SÍMBOLOS

req/s	Requisições por segundo
p_{50}	Percentil 50 (Mediana da latência)
p_{95}	Percentil 95 de latência
p_{99}	Percentil 99 de latência
$R\$$	Real (moeda brasileira)
$US\$$	Dólar americano
GHz	Gigahertz
MHz	Megahertz
GB	Gigabyte
MB	Megabyte

SUMÁRIO

1	INTRODUÇÃO	7
2	JUSTIFICATIVA	9
3	OBJETIVOS	10
3.1	OBJETIVO GERAL	10
3.2	OBJETIVOS ESPECÍFICOS	10
4	REFERENCIAL TEÓRICO	11
4.1	Escalabilidade em Sistemas <i>web</i>	11
4.2	Arquitetura Monolítica	11
4.3	Arquiteturas Distribuídas e Microserviços	12
4.4	Linguagens de Programação e Modelos de Execução	13
5	PROCEDIMENTOS METODOLÓGICOS	14
5.1	Definição do Ambiente de Testes	14
5.2	Implementação dos Modelos Experimentais	14
5.2.1	Modelos Monolíticos (<i>Java</i> e <i>Rust</i>)	14
5.2.2	Modelo Distribuído (<i>Java</i>)	14
5.2.3	Camada de Dados e Persistência	15
5.3	Protocolo Experimental e Coleta de Dados	15
5.3.1	Controle de Recursos	15
5.3.2	Execução dos Testes e Métricas	15
6	RESULTADOS ESPERADOS	16
7	ORÇAMENTO	17
8	CRONOGRAMA	18
	REFERÊNCIAS	19

1 INTRODUÇÃO

No período pós-pandemia, observou-se um crescimento inevitável dos sistemas *web* e um aumento substancial na demanda por escalabilidade e pela manutenção da disponibilidade dessas aplicações. Esse cenário é refletido nos indicadores do setor; de acordo com a Associação Brasileira de Comércio Eletrônico (ABCOMM, 2024), o faturamento do *e-commerce* no país atingiu a marca de R\$ 204,3 bilhões, com um volume superior a 414 milhões de pedidos realizados. Esse cenário de expansão acelerada não foi apenas uma transição gradual, mas uma resposta de sobrevivência ao contexto global. Segundo Carreiro e Nose (2023), o avanço do mercado eletrônico brasileiro foi catalisado pela COVID-19, o que forçou as empresas a adotarem estratégias de aprimoramento de suas infraestruturas digitais para suportar o novo volume de transações e acessos.

Entretanto, a transição para sistemas capazes de suportar esse volume massivo de transações apresenta desafios técnicos complexos. A dificuldade em escalar aplicações reside, muitas vezes, na identificação e mitigação de gargalos de desempenho que surgem sob alta carga. Segundo Gregol e Schutz (2013), não existe uma fórmula única para a escalabilidade, sendo indispensável a análise rigorosa dos recursos necessários para atender múltiplos usuários simultâneos. Segundo Mendes (2021), a arquitetura monolítica é frequentemente a opção inicial devido ao seu custo reduzido, sendo ideal para a fase de descoberta de um produto. Contudo, a autora ressalta que esse modelo tende a perder sua capacidade de manutenção e evolução conforme o sistema cresce, o que leva muitos desenvolvedores a considerá-lo uma "arquitetura de sacrifício", que deve ser substituída por modelos mais sustentáveis após a validação do domínio de negócio e dos requisitos reais de escalabilidade.

Diante da necessidade de evolução para modelos mais robustos, o debate arquitetural centra-se na premissa de que não existe uma solução perfeita, exigindo que as escolhas sejam baseadas em *trade-offs* específicos entre simplicidade e poder de escala. Conforme analisa Carvalho (2022), embora a arquitetura monolítica se destaque pela menor complexidade de desenvolvimento em diversos cenários, ela apresenta limitações na eficiência da escalabilidade, pois exige a replicação de todo o sistema para expandir componentes específicos.

Em contrapartida, arquiteturas distribuídas, como os microsserviços, oferecem maior flexibilidade e resiliência, permitindo o isolamento de falhas e a escalabilidade independente de serviços conforme a demanda. Contudo, Carvalho (2022) ressalta que esses benefícios impõem um aumento significativo na complexidade de infraestrutura e na gestão da comunicação entre serviços. Assim, a decisão por um modelo distribuído deve considerar se a complexidade e o tamanho do sistema justificam o esforço adicional e os custos operacionais envolvidos, sob o risco de introduzir dificuldades desnecessárias ao projeto.

Somado ao dilema arquitetural, a escolha da linguagem de programação atua como fator determinante na eficiência da escalabilidade. Embora o *Java* seja consolidado em sistemas corporativos por sua robustez, Rezzaghi e Silva (2023) apontam que sua performance em larga

escala pode sofrer impactos devido à dependência do *garbage collector*. Em contrapartida, linguagens modernas como o *Rust* utilizam o sistema de *Ownership*, eliminando a necessidade de um coletor de lixo e garantindo, conforme demonstram os testes dos mesmos autores, uma performance superior sob cargas elevadas de processamento. Tais evidências sugerem que a eficiência de uma linguagem de baixo nível pode mitigar limitações inerentes ao modelo monolítico, oferecendo uma alternativa de escala viável antes da transição para estruturas distribuídas.

Apesar da relevância dessas discussões, observa-se que a maioria dos estudos disponíveis foca ou exclusivamente em aspectos arquiteturais de alto nível ou em comparações de performance em nível unitário de código. Nota-se uma escassez de análises empíricas que integrem, em um mesmo experimento, as variáveis de arquitetura e linguagem de programação dentro de um sistema *web* completo, submetido a condições reais de estresse de rede e concorrência.

Diante dessa lacuna, este trabalho tem como objetivo realizar um estudo comparativo utilizando um serviço de encurtamento de URLs como objeto experimental. A escolha desse serviço justifica-se por sua natureza intensiva em operações de entrada e saída (*I/O*) e pela necessidade de baixa latência, características que o tornam um modelo ideal para isolar o impacto da arquitetura e da linguagem no desempenho global. Ao submeter três implementações distintas (monolítico *Java*, monolítico *Rust* e distribuído *Java*) a cargas de até 1.500 requisições por segundo, busca-se fornecer dados quantitativos que auxiliem na tomada de decisão sobre o momento ideal para a transição arquitetural ou a adoção de novas tecnologias para garantir a escalabilidade.

2 JUSTIFICATIVA

O crescimento acelerado de aplicações *web*, com o mercado global de aplicações *SaaS* projetado para atingir US\$ 1.131,52 bilhões até 2032 (Fortune Business Insights, 2025), tem levado desenvolvedores e equipes de tecnologia a enfrentarem um dilema recorrente: como escalar um sistema de forma eficiente sem comprometer desempenho, estabilidade e custo operacional. Quando o volume de usuários e transações aumenta, surgem incertezas sobre qual caminho seguir. Muitas vezes não há clareza se o gargalo pode ser resolvido com ajustes na arquitetura atual, se é necessário adotar uma arquitetura distribuída ou até mesmo reescrever o sistema em outra linguagem de programação. Essa falta de referências práticas e comparativas dificulta a tomada de decisões técnicas fundamentadas e pode resultar em custos elevados com reescritas completas que consomem meses de desenvolvimento, migrações para microsserviços que aumentam a complexidade sem ganhos reais, ou manutenção de arquiteturas monolíticas que colapsam sob alta demanda.

Nesse contexto, este trabalho se justifica pela necessidade de produzir uma análise concreta sobre como diferentes abordagens arquiteturais e como diferentes linguagens se comportam sob alta carga. Embora existam discussões teóricas sobre monólitos, microsserviços e escalabilidade, há pouca experimentação prática que combine três dimensões de análise simultaneamente: (1) comparação entre linguagens de programação (*Java* vs. *Rust*), (2) comparação entre arquiteturas (monolítica vs. distribuída), e (3) avaliação sob condições de carga realistas e controladas. A construção de um encurtador de URLs como caso de estudo permite simular um cenário consistente de uso intensivo, utilizando métricas objetivas como latência, *throughput* e consumo de recursos para avaliar o comportamento do sistema.

Dessa forma, o projeto contribui para preencher uma lacuna prática na literatura e no cotidiano do desenvolvimento de *software*, oferecendo dados comparativos que auxiliam profissionais e estudantes a compreenderem melhor os limites e vantagens de diferentes arquiteturas. Do ponto de vista acadêmico, o trabalho contribui com uma metodologia replicável de avaliação de desempenho que pode ser adaptada para outros domínios de aplicação, além de gerar um conjunto de dados experimentais que poderá servir como referência para pesquisas futuras em engenharia de *software* e arquitetura de sistemas. O estudo também apoia decisões mais assertivas em futuras implementações, evitando reescritas desnecessárias e promovendo um entendimento mais sólido sobre como sistemas *web* respondem ao crescimento.

3 OBJETIVOS

3.1 OBJETIVO GERAL

Investigar como diferentes abordagens arquiteturas e escolhas de linguagem de programação influenciam a escalabilidade de sistemas *web*, comparando um modelo monolítico em *Java*, um monolítico em *Rust* e uma arquitetura distribuída em *Java*, utilizando como caso prático um serviço de encurtamento de URLs.

3.2 OBJETIVOS ESPECÍFICOS

- Implementar as três variações do serviço de encurtamento de URLs (*Java* monólito, *Rust* monólito e *Java* distribuído) com funcionalidades equivalentes.
- Executar testes de estresse progressivos para identificar limites de escalabilidade e pontos de ruptura de cada implementação.
- Analisar métricas de desempenho focadas em latência (p_{50} , p_{95} e p_{99}), *throughput* e taxas de erro.
- Avaliar os *trade-offs* entre desempenho, complexidade e custo operacional das arquiteturas e linguagens testadas.
- Disponibilizar o código-fonte e o ambiente experimental para garantir a replicabilidade do estudo.

4 REFERENCIAL TEÓRICO

4.1 ESCALABILIDADE EM SISTEMAS WEB

No cenário contemporâneo de sistemas *web*, a escalabilidade não é apenas uma característica desejável, mas um requisito crítico para a sobrevivência de aplicações sob demanda variável. Segundo Kleppmann (2017), a escalabilidade é definida como a capacidade de um sistema de continuar operando de maneira eficiente à medida que a carga de trabalho — seja em volume de requisições ou tamanho de dados — aumenta significativamente. O autor enfatiza que a escalabilidade não deve ser vista como uma propriedade binária, mas como um conjunto de estratégias para lidar com o crescimento da carga sem comprometer o desempenho.

Para atingir esse objetivo, a literatura técnica estabelece duas abordagens principais, cada uma com seus respectivos *trade-offs*:

- **Escalabilidade Vertical (*Scaling Up*):** Consiste no incremento de recursos em um único nó, como o acréscimo de *CPU* e memória *RAM*. Conforme apontam Bass, Clements e Kazman (2012), embora seja uma estratégia de baixa complexidade de implementação, ela encontra limites físicos e financeiros no *hardware*, além de criar um ponto único de falha (*single point of failure*).
- **Escalabilidade Horizontal (*Scaling Out*):** Envolve a distribuição da carga através da adição de novos nós em um ecossistema distribuído. Tanenbaum e Van Steen (2017) destacam que, apesar de oferecer um teto de crescimento teoricamente ilimitado e maior tolerância a falhas, essa abordagem introduz uma complexidade considerável na infraestrutura e na rede, exigindo mecanismos de balanceamento de carga e sincronização de dados.

Dessa forma, a escolha entre escalar verticalmente através da otimização de recursos locais ou horizontalmente através da distribuição sistêmica estabelece a base para discutir as estruturas de organização de *software*. Para analisar como tais estratégias são aplicadas na prática, torna-se fundamental explorar as características das arquiteturas monolíticas e distribuídas, tópicos abordados nas seções seguintes.

4.2 ARQUITETURA MONOLÍTICA

A arquitetura monolítica é o padrão tradicional de construção de sistemas, no qual todos os componentes funcionais de uma aplicação — como a lógica de negócio, o acesso a dados e a interface de usuário — são integrados e implantados como uma única unidade lógica (*single deployment unit*). Segundo Richardson (2018), nesse modelo, a aplicação é empacotada em um único arquivo executável (como um arquivo *.jar* no ecossistema *Java* ou um binário compilado em *Rust*), o que simplifica o fluxo inicial de desenvolvimento e operação.

A principal característica do monólito é o compartilhamento de recursos internos. Todas as chamadas entre funções e módulos ocorrem via pilha de chamadas da linguagem (*stack*) e memória compartilhada, eliminando o *overhead* de rede e a necessidade de serialização de dados, o que resulta em uma latência interna mínima. Conforme destaca Newman (2015), a simplicidade de testar e monitorar um monólito é um de seus maiores pontos positivos, pois não há a necessidade de lidar com a complexidade de sistemas distribuídos, como a consistência eventual e falhas parciais de rede.

Entretanto, discussões arquiteturais apontam que o crescimento desordenado de um monólito pode levar ao padrão conhecido como *Big Ball of Mud* (Grande Bola de Lama). Fowler (2015) argumenta que, embora existam benefícios na simplicidade inicial, o aumento da base de código dificulta o entendimento sistêmico, aumenta o tempo de compilação (*build time*) e impede a escalabilidade granular. No monólito, para escalar uma funcionalidade específica, é necessário replicar toda a aplicação, consumindo recursos de *CPU* e memória de componentes que não estão sob estresse.

Atualmente, surge como uma evolução desse padrão o conceito de Monólito Modular. Segundo Richardson (2018), um monólito bem estruturado, com módulos claramente definidos e baixo acoplamento, pode sustentar altas cargas de trabalho antes que a migração para microsserviços se torne tecnicamente justificável. Essa perspectiva é fundamental para o presente trabalho, pois permite avaliar se a utilização de linguagens de alto desempenho, como o *Rust*, pode estender a vida útil e a eficiência de uma arquitetura monolítica, adiando a necessidade de adoção de estruturas distribuídas mais complexas.

4.3 ARQUITETURAS DISTRIBUÍDAS E MICROSERVIÇOS

A arquitetura distribuída fundamenta-se na composição de componentes autônomos que, embora executados em processos ou máquinas distintas, operam de maneira coordenada. Segundo Tanenbaum e Van Steen (2017), a motivação para a distribuição reside na organização de recursos de forma que o sistema suporte falhas parciais e apresente transparência ao usuário, funcionando como uma unidade coerente apesar da separação física de seus elementos.

No âmbito do desenvolvimento de sistemas orientados a serviços, a abordagem de microsserviços materializa esses conceitos ao decompor a aplicação em unidades funcionais mínimas. Newman (2015) destaca que a característica central dessa estratégia é a capacidade de implantação independente (*independent deployability*). Essa autonomia permite que cada componente do sistema evolua, seja atualizado e, fundamentalmente, seja replicado sem exigir a coordenação ou a reimplantação de outros serviços do ecossistema.

A viabilização dessa estrutura exige componentes de orquestração e roteamento. Richardson (2018) propõe o uso do padrão *API Gateway* como uma camada de abstração que centraliza o tráfego externo e direciona as requisições para os serviços pertinentes. Essa organização é o que permite a implementação da escalabilidade seletiva: em cenários de carga assimétrica, é possível alocar mais instâncias para serviços de alta demanda (como a consulta

de URLs) enquanto serviços de menor tráfego (como a criação de URLs) permanecem com recursos reduzidos.

Portanto, a arquitetura distribuída neste trabalho é caracterizada pela segregação funcional entre os serviços de escrita e leitura. Essa configuração visa isolar o impacto das chamadas de rede e do balanceamento de carga, permitindo analisar se a flexibilidade em escalar o serviço de redirecionamento de forma granular oferece ganhos de vazão (*throughput*) que justifiquem a complexidade adicional na infraestrutura.

4.4 LINGUAGENS DE PROGRAMAÇÃO E MODELOS DE EXECUÇÃO

A escolha da linguagem de programação exerce um impacto determinante na eficiência de um sistema sob carga, uma vez que diferentes tempos de execução (*runtimes*) gerenciam os recursos de *hardware* de maneiras distintas. No contexto deste trabalho, o contraste entre *Java* e *Rust* representa o embate entre o modelo de gerenciamento automático de memória via Máquina Virtual e o modelo de segurança de memória em tempo de compilação.

O ecossistema *Java* fundamenta-se na *Java Virtual Machine (JVM)*, que utiliza a técnica de compilação *Just-In-Time (JIT)* para otimizar o código em tempo de execução. Segundo Goetz (2006), uma das características centrais da *JVM* é o *Garbage Collector (GC)*, um mecanismo automático que identifica e remove objetos não utilizados da memória *heap*. Embora o *GC* simplifique o desenvolvimento, Goetz ressalta que ele pode introduzir pausas imprevisíveis na execução (*stop-the-world pauses*), o que, em cenários de alta vazão, pode elevar a latência dos percentis de cauda (*tail latency*).

Em contrapartida, a linguagem *Rust* propõe um modelo de alto desempenho sem a necessidade de um *Garbage Collector*. Klabnik e Nichols (2018) explicam que o *Rust* utiliza um sistema de *Ownership* (posse) e *Borrowing* (empréstimo), cujas regras são validadas estritamente durante a compilação. Esse modelo permite que o programador tenha controle total sobre a alocação e desalocação de memória sem os riscos de falhas de segmentação, resultando em uma *performance* determinística e no uso de abstrações de custo zero.

Portanto, a comparação entre essas linguagens no experimento de escalabilidade visa observar como o comportamento do *GC* do *Java* se comporta frente ao modelo determinístico do *Rust* sob uma carga constante de 1.500 *req/s*. Enquanto o *Java* representa a maturidade e a produtividade do mercado corporativo, o *Rust* surge como uma alternativa para sistemas onde a eficiência e a previsibilidade do uso de *CPU* e memória são requisitos inegociáveis.

5 PROCEDIMENTOS METODOLÓGICOS

5.1 DEFINIÇÃO DO AMBIENTE DE TESTES

Para garantir a replicabilidade e o isolamento das variáveis de arquitetura e linguagem, os testes serão conduzidos em um ambiente controlado via virtualização, utilizando *Docker* e *Docker Compose*. O ambiente de execução consistirá na seguinte configuração:

- **Sistema Operacional:** *Ubuntu 24.04.3 LTS*;
- **Hardware:** Processador *Intel® Core™ i5-13420H* (8 núcleos, 12 *threads*, até 4,60 *GHz*) e 16 *GB* de memória *RAM DDR5 4800 MHz*;
- **Armazenamento:** *SSD 512 GB M.2 PCIe Gen4*;

5.2 IMPLEMENTAÇÃO DOS MODELOS EXPERIMENTAIS

Para a comparação empírica, serão desenvolvidas três versões funcionais do encurtador de URLs, todas expondo os mesmos *endpoints*: *POST* para criação e *GET* para redirecionamento.

5.2.1 Modelos Monolíticos (*Java* e *Rust*)

Os modelos monolíticos serão construídos como unidades únicas de execução (*single process*). Ambas as versões compartilharão a mesma lógica de negócio, mas com ecossistemas distintos:

- **Monólito *Java*:** Implementado com *Spring Boot*, utilizando o servidor embarcado *Tomcat* e gerenciamento de memória via *Garbage Collector*.
- **Monólito *Rust*:** Implementado com *Actix-web*, utilizando o *runtime* assíncrono *Tokio* e gerenciamento de memória determinístico via *Ownership*.

5.2.2 Modelo Distribuído (*Java*)

A versão distribuída será implementada em *Java*, seguindo o princípio de segregação de responsabilidades em *containers* isolados:

- ***API Gateway (Nginx)*:** Ponto de entrada que realiza o roteamento baseado no método *HTTP*.
- ***Writer Service*:** Serviço *Spring Boot* dedicado à criação e persistência de URLs.
- ***Reader Service*:** Serviço *Spring Boot* focado em alta vazão de redirecionamentos, configurado com três réplicas balanceadas pelo *Gateway*.

5.2.3 Camada de Dados e Persistência

Todas as versões utilizarão a mesma infraestrutura de dados para garantir a isonomia dos testes:

- **PostgreSQL:** Atuando como banco de dados relacional definitivo com índice único sobre a coluna de códigos encurtados. O gerenciamento de conexões será equalizado entre as versões via *connection pools* (*HikariCP* e *SQLx*).

5.3 PROTOCOLO EXPERIMENTAL E COLETA DE DADOS

5.3.1 Controle de Recursos

Cada *container* terá seus recursos limitados via *Docker Compose*:

- **Limits:** Teto máximo de 0,5 *CPU* e 512 *MB RAM* por instância.
- **Reservations:** Reserva mínima imediata de recursos ao iniciar o *container*.

5.3.2 Execução dos Testes e Métricas

Utilizando a ferramenta *k6*, os testes seguirão um perfil progressivo (*ramping-up*) de 10 até 1.500 *req/s*. Serão coletadas métricas de:

1. **Latência:** Percentis p_{50} , p_{95} e p_{99} .
2. **Throughput:** Requisições processadas com sucesso por segundo.
3. **Taxa de Erro:** Percentual de falhas ou *timeouts*.

6 RESULTADOS ESPERADOS

Com a realização deste estudo, espera-se obter um conjunto de dados empíricos que permitam uma análise objetiva sobre os limites de escalabilidade das tecnologias propostas. As principais hipóteses a serem validadas são:

- **Desempenho do *Rust*:** Espera-se que o monólito em *Rust* apresente a menor latência média e, principalmente, uma maior estabilidade nos percentis de cauda (p_{95} e p_{99}), devido ao seu gerenciamento de memória determinístico e ausência de interrupções por *Garbage Collection*.
- **Escalabilidade Granular do Modelo Distribuído:** Espera-se que a arquitetura distribuída em *Java* demonstre maior resiliência em cargas extremas (próximas a 1.500 *req/s*) ao permitir a replicação seletiva do serviço de redirecionamento, superando o *throughput* do monólito *Java*.
- **Custo de Comunicação:** Espera-se identificar que, em cargas baixas, os monólitos superem a arquitetura distribuída em tempo de resposta, evidenciando o *overhead* introduzido pela rede e pelo balanceamento de carga do *Gateway*.
- **Consumo de Recursos:** Antecipa-se que a implementação em *Rust* apresente um consumo de memória significativamente inferior às versões em *Java*, mesmo sob estresse máximo.

Ao final, o estudo pretende entregar um guia comparativo que auxilie na tomada de decisão entre otimizar a linguagem de um sistema ou migrar para uma arquitetura distribuída.

7 ORÇAMENTO

A execução deste projeto não demanda recursos financeiros externos ou fomento de agências de pesquisa. O desenvolvimento dos artefatos e a condução dos testes experimentais serão realizados sob as seguintes condições:

- **Hardware:** Utilização de equipamento pessoal (computador) para o desenvolvimento e simulação do ambiente de testes via *Docker*.
- **Software:** Uso exclusivo de tecnologias de código aberto (*open-source*), incluindo as linguagens *Java* e *Rust*, os bancos de dados *PostgreSQL*, e a ferramenta de teste *k6*.
- **Infraestrutura:** Os custos de energia elétrica e acesso à internet são de responsabilidade do pesquisador.

REFERÊNCIAS

- ABCOMM. **Principais Indicadores do e-Commerce**. 2024. Acesso em: 05 jan. 2026. Disponível em: <<https://dados.abcomm.org/numeros-do-ecommerce-brasileiro>>.
- BASS, L.; CLEMENTS, P.; KAZMAN, R. **Software Architecture in Practice**. 3. ed. Upper Saddle River: Addison-Wesley, 2012.
- CARREIRO, A. A. S.; NOSE, E. T. O avanço do e-commerce brasileiro pré e pós pandemia. **RIT – Revista Inovação Tecnológica**, v. 13, n. 1, 2023. ISSN 2179-2895.
- CARVALHO, M. D. d. Migração de sistemas erp monolíticos para a arquitetura de microserviços. **Abakos**, Belo Horizonte, v. 1, n. 1, 2022. ISSN 2316-9451.
- Fortune Business Insights. **Software as a Service (SaaS) Market Size, Share & Industry Analysis, 2025–2032**. 2025. Acesso em: 03 dez. 2025. Disponível em: <<https://www.fortunebusinessinsights.com/pt/software-as-a-service-saas-market-102222>>.
- FOWLER, M. **MonolithFirst**. 2015. Disponível em: <<https://martinfowler.com/bliki/MonolithFirst.html>>. Acesso em: 04 fev 2026.
- GOETZ, B. *et al.* **Java Concurrency in Practice**. Boston: Addison-Wesley Professional, 2006.
- GREGOL, R. E.; SCHUTZ, F. Recursos de escalabilidade e alta disponibilidade para aplicações web. **Revista Eletrônica Científica Inovação e Tecnologia**, p. 28–30, 2013. ISSN 2175-1846.
- KLABNIK, S.; NICHOLS, C. **The Rust Programming Language**. San Francisco: No Starch Press, 2018.
- KLEPPMANN, M. **Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems**. Sebastopol: O'Reilly Media, 2017.
- MENDES, I. S. **Arquitetura monolítica vs microserviços: uma análise comparativa**. Monografia (Trabalho de Conclusão de Curso (Graduação em Engenharia de Software)) — Universidade de Brasília, Brasília, DF, 2021.
- NEWMAN, S. **Building Microservices: Designing Fine-Grained Systems**. Sebastopol: O'Reilly Media, 2015.
- REZZAGHI, L. B.; SILVA, A. M. Programação concorrente em rust e java: uma análise comparativa de segurança, produção e performance. In: **Anais do WCF**. [S.l.: s.n.], 2023. v. 10.
- RICHARDSON, C. **Microservices Patterns: With examples in Java**. Shelter Island: Manning Publications, 2018.
- TANENBAUM, A. S.; STEEN, M. V. **Distributed Systems**. 3. ed. Amsterdam: Pearson Education, 2017.